

DevOps: DogOps - ספר פרויקט גמר

שם המנחה: DevOps **מוגש על ידי:** [הכנס שם מלא] **תעודת זהות:** [הכנס ת.ז.] **מכללה:** אורט סינגאלובסקי **מגמה** [הכנס שם מנחה]

תוכן עניינים

1. הצהרת סטודנטים
2. מבוא לפרויקט
3. מטרת המערכת והאפליקציה
4. ארכיטקטורת המערכת
5. הסבר על כל כלי
6. CI/CD Pipeline ו-GitFlow תהליך העבודה
7. Terraform - Infrastructure as Code
8. Docker - קונטיינריזציה
9. Kubernetes ו-GitOps
10. ניטור (Monitoring) ו-Observability
11. בעיות צפויות ופתרונות
12. וצילומי מסך (Code Snippets) תמונות קוד
13. סיכום אישי

הצהרת סטודנטים

אני החתום מטה, מצהיר בזאת כי כל פרויקט הגמר המוגש בספר זה, הינו פרי עבודתי בלבד, על בסיס הנחייתו של המנחה ותוך הסתמכות על מקורות הידע והמידע האחרים המצויים בביבליוגרפיה המובאת בסיום ספר זה. אני מודע לאחריות שאני מקבל על עצמי ע"י חתימתי על הצהרה זאת, שכל הנאמר בה הינה אמת ורק אמת.

חתימה: _____

מבוא לפרויקט

הוא תרבות וגישה טכנולוגית שנועדה לגשר על הפערים ההיסטוריים (Development & Operations) -DevOps תחום ה בעבר, מפתחים היו כותבים קוד ו"זורקים" אותו לצוותי התפעול כדי שיפרסו אותו. (IT) בין צוותי הפיתוח לצוותי התפעול תהליך זה היה איטי, מועד לטעויות, וגרם לחיכוך רב. (Deployment) בשרתים.

הלכה למעשה. הפרויקט DevOps-המיישם את עקרונות ה (End-to-End) פרויקט זה מציג פתרון מקיף וקצה-לקצה ולבצע אוטומציה מלאה משלב כתיבת (Microservices) מדגים כיצד ניתן לקחת אפליקציה המורכבת ממספר שכבות Production. הקוד, דרך בדיקות, אריזה, יצירת תשתיות ענן, ועד לפריסה שקופה, בטוחה וניתנת לניטור בסביבת

מטרת המערכת והאפליקציה

מדובר באפליקציה בעלת **DogOps** מטרת הפרויקט היא להקים סביבת ענן מודרנית ואוטומטית עבור אפליקציית קלאסית Tier-ארכיטקטורת 3

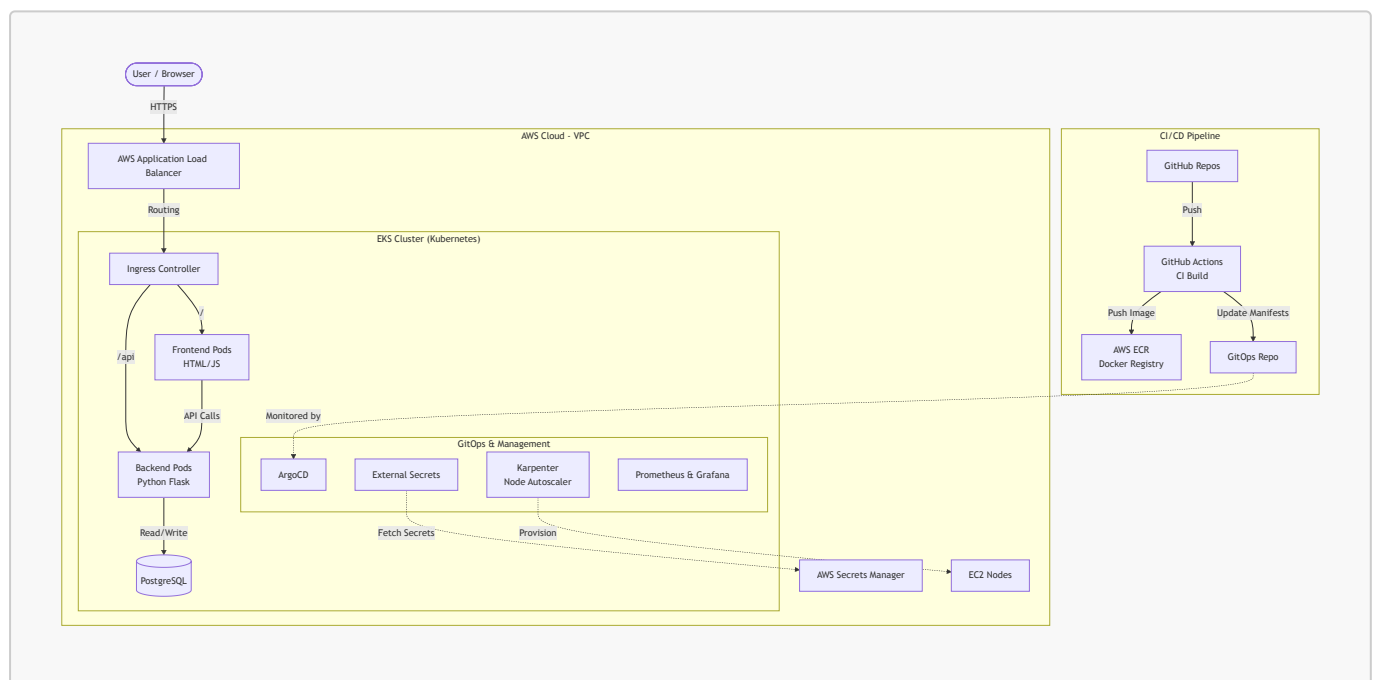
1. **Frontend (צד לקוח):** השכבה הזו (Vanilla) נקי JavaScript ו-CSS ו-HTML ממשק משתמש הכתוב ב: (צד לקוח).
2. **Backend (צד שרת):** שרת זה מטפל (Flask באמצעות ספריית Python מנוע האפליקציה הכתוב בשפת: (צד שרת)). ומקשר בין צד הלקוח לבסיס הנתונים API-בלוגיקה העסקית, מנהל את ממשקי ה.
3. **Database (בסיס נתונים):** הממשק לשמירת המידע בצורה רלציונית, בטוחה PostgreSQL בסיס נתונים מסוג: (בסיס נתונים). ושרידה.

המטרות הטכנולוגיות של הפרויקט:

- מפעיל שרשרת אוטומטית של בנייה, בדיקה ופריסה (Push) כל שינוי קוד: **(CI/CD) אוטומציה מלאה**.
- באמצעות קוד ללא (EKS רשתות, הרשאות, קלאסטר) AWS-הקמה של כל סביבת ה: **(IaC) תשתית כקוד**. התערבות ידנית בממשק המשתמש של אמזון.
- את השינויים אוטומטית ומסנכרן (Pull) כך שהקלאסטר מושך, Git ניהול מצב הקלאסטר מתוך מאגר: **GitOps**. את עצמו.
- באמצעות כלים (Autoscaling) יכולת לגדול ולקטון אוטומטית: **(High Availability) גמישות ועמידות**. Karpenter מתקדמים כמו.

ארכיטקטורת המערכת

Kubernetes ועל סביבת AWS הארכיטקטורה מבוססת לחלוטין על שירותי הענן של



הסבר על כל כלי

1. AWS (Amazon Web Services)

ספקית הענן המובילה בעולם שעליה מתארח הפרויקט. בפרויקט נעשה שימוש ב:

- **VPC:** (NAT/IGW) ציבוריים ופרטיים, נתבים Subnets, ניהול הרשת הפרטית.
- **EKS (Elastic Kubernetes Service):** שירות ניהול הקלאסטר של קוברנטיס.
- **S3:** (Terraform של State-כגון קבצי ה) אחסון אובייקטים.
- **IAM / IRSA:** ניהול הרשאות מאובטח למשאבים ולפודים.

2. Docker

שמכילה (VM) בניגוד למכונה וירטואלית. (Containers) "כלי לאריזת האפליקציה והתלויות שלה לתוך "קונטיינרים של המארח, מה שהופך אותו לקל משקל, מהיר ועקבי בין (Kernel) מערכת הפעלה שלמה, קונטיינר משתמש בליבה סביבות ("עובד אצלי, יעבוד בכל מקום").

3. Kubernetes (K8s)

וניהול של אלפי קונטיינרים. אחראית על (Orchestration) מערכת קוד פתוח לתזמור:

- (פוד שנופל מוקם מחדש - Self-healing) התאוששות מאסון.
- (Load Balancing) חלוקת עומסים.
- (Scaling) חשיפת שירותים והגדרת כמות רפליקות.

4. Terraform

המתאר HCL בשפת (Declarative) מאפשר לכתוב קוד הצהרתי. (Infrastructure as Code) כלי ליצירת תשתיות כקוד בצורה מדויקת, עקבית וניתנת לשחזור ולמעקב גרסאות AWS-דואג להקים אותה ב-Terraform את התשתית הרצויה, ו

5. GitHub Actions

אוטומטיים המריצים סקריפטים (Pipelines) "מאפשר ליצור "צינורות תהליך. GitHub המובנה בתוך CI/CD שירות קוד חדש (Push) ופעולות בכל פעם שמפתחים דוחפים.

6. ArgoCD (GitOps)

מאזין למאגר גיט שמכיל את קבצי ArgoCD. GitOps-לקוברנטים המיישם את גישת ה Continuous Delivery כלי (Sync) וברגע שהוא מזהה שינוי (למשל, עדכון גרסת אימג'), הוא מסנכרן, (Manifests/Helm) התצורה של קוברנטים אוטומטית את הקלאסטר למצב הרצוי המוגדר בקוד.

7. כלים מתקדמים (Karpenter & External Secrets)

- **Karpenter:** שנועד לשפר את הגמישות של קוברנטים. הוא מזהה פודים שממתינים AWS פרויקט קוד פתוח של לשרת בגדלים מדויקים ובעלות אופטימלית, במקום להשתמש EC2 ומקים עבורם תוך שניות שרתי (Pending) לשרת סטטיים. זהו כלי מתקדם מאוד המבטיח ניצולת משאבים מושלמת Node Groups-ב.
- **External Secrets Operator (ESO):** כמו) במקום לשמור סיסמאות בקבצי הקוד, מערכת זו מושכת סודות (API או לממשקי DB-סיסמאות ל ומזריקה אותם לקלאסטר בצורה AWS Secrets Manager-ישירות מ (API או לממשקי DB-סיסמאות ל מאובטחת).

CI/CD Pipeline-תהליך העבודה

(Repositories) וניהול 3 מאגרים מופרדים **GitFlow** בפרויקט יושמה מתודולוגיית עבודה מבוססת:

1. **devops-dogops-app** (Frontend, Backend) מכיל את קוד המקור של האפליקציה: Charts.
2. **devops-dogops-infra** שמקים את תשתית הענן Terraform-מכיל את קוד ה: Charts.
3. **devops-dogops-gitops** (Environments - Prod, Staging, Test) מכיל את תצורת הסביבות ה-ArgoCD.

המלאה CI/CD-זרימת ה (The Pipeline)

(GitHub Actions דרך) הבא Pipeline-לענף הראשי במאגר האפליקציה, מופעל ה `git push` כאשר מפתח מבצע:

1. **Linting & Testing:** כדי לוודא שאין שגיאות (Python-ל `pylint` למשל) הקוד נבדק אל מול תקני כתיבה.
2. **Docker Build:** Backend-ואחד עבור ה Frontend-אחד עבור ה - (Images) המערכת בונה שני קונטיינרים.
3. **Push to Registry:** (Amazon ECR) האימג'ים נדחפים למאגר התמונות המאובטח של אמזון (Tag / Commit SHA) בנייה.
4. **Update GitOps Repo:** GitOps-למאגר ה Checkout סקריפט אוטומטי עושה: Commit החדש שנוצר, ומבצע אליו Tag-של הסביבה הרלוונטית עם ה `values.yaml`.
5. **ArgoCD Sync:** GitOps-שרץ בתוך הקלאסטר מזהה שינוי במאגר ה ArgoCD-רכיב ה: הוא משווה את המצב. Rolling Update) הנוכחי למצב הרצוי, ומתחיל תהליך פריסה אוטומטי וזהיר שמחליף את הפודים הישנים (Locking) עם מנגנון נעילה, `S3 Bucket`-בפרויקט נשמר מרחוק ב State-ניהול ה של טרפורם מכיל את מיפוי המשאבים הקיים מול הקוד. כדי לאפשר עבודת צוות, ה State-קובץ ה: **State-ניהול** ה State DynamoDB המנוהל דרך (Locking) עם מנגנון נעילה, `S3 Bucket`-בפרויקט נשמר מרחוק ב State.

Terraform - Infrastructure as Code

AWS-במקום לייצר משאבים ידנית ב Terraform. התשתית כולה מנוהלת דרך תיקיית המודולים והסביבות של Management Console, (Dev, Staging, Prod) יצרנו מודול תשתית אחיד שניתן לשכפל לסביבות שונות.

-של טרפורם מכיל את מיפוי המשאבים הקיים מול הקוד. כדי לאפשר עבודת צוות, ה State-קובץ ה: **State-ניהול** ה State DynamoDB המנוהל דרך (Locking) עם מנגנון נעילה, `S3 Bucket`-בפרויקט נשמר מרחוק ב State.

מודולים מרכזיים בפרויקט:

- **VPC Module:** Private/Public Subnets, NAT Gateways ו-Route Tables ליצירת רשת עם Terraform שימוש במודול הרשמי של קהילת.
- **EKS Module:** הקמת הקלאסטר עצמו, ניהול תעודות, והגדרת הרשאות הגישה.
- **Helm Provider:** אשר מתקינים אוטומטית שירותי תשתית על "Helm Releases" הוגדרו Terraform-בתוך ה: (Prometheus), כלי ניטור, ArgoCD, הקלאסטר מיד עם הקמתו: איגרגס קונטרולר (ESO) וניהול סודות.

ה: Terraform-דוגמה מקוד ה (Prod סביבת):

```
module "infrastructure" {
  source = "../../modules/infrastructure"

  environment      = "prod"
  vpc_cidr          = "10.3.0.0/16"
  node_instance_type = "t3.medium"

  domain_name      = "dogops.co"
}
```

Docker - קונטיינריזציה

כגון Docker, בבניית תמונות Best Practices משלו. הקפדנו על עקרונות `Dockerfile` לכל רכיב באפליקציה יש ריצה ללא הרשאות) כדי להקטין את נפח האימג' הסופי ולשפר את רמת האבטחה Multi-stage builds-שימוש ב (root).

Dockerfile של ה Backend (Python/Flask): דוגמה ל

```
# Stage 1: Build dependencies
FROM python:3.11-slim as builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# Stage 2: Final minimal image
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
# Run as non-root user for security
RUN useradd -m appuser && chown -R appuser /app
USER appuser

EXPOSE 5000
CMD ["flask", "run", "--host=0.0.0.0"]
```

Kubernetes I-GitOps

המבנה בקורנטיס

- **Deployments:** כל רכיב (Frontend, Backend) מנוהל על ידי Deployment המוגדרת שכמות הפודים המוגדרת מנוהל על ידי (Replicas) פועלת תמיד.
- **Services:** מקבל תעבורה Backend-וה, Ingress-מקבל תעבורה מה Frontend-חשיפה פנימית של הפודים. ה: (שמירה על אבטחה) Frontend-רק מה.
- **Ingress:** מהעולם החיצון HTTP/HTTPS-המנתב את תעבורת ה, (AWS ALB) שער הכניסה של האפליקציה: (Path-based routing) לתוך הקלאסטר על בסיס נתיבים.
- **Helm Charts:** המאפשרות להזריק פרמטרים שונים (כמו כתובת מסד Helm האפליקציה נארזה לתוך תבניות הנתונים או גרסת האימג') לכל סביבה בנפרד.

GitOps באמצעות ArgoCD ApplicationSets

עבור ArgoCD-במקום ליצור ידנית הגדרה ב **ApplicationSets**-אחד האלמנטים המתקדמים בפרויקט הוא השימוש ב ומייצר דינמית את **environments**-כל סביבה וכל אפליקציה, נכתב קובץ תבנית אחד שסורק את תיקיית ה (Scalability) שמאפשר צמיחה וניהול קלים Enterprise זהו יישום ברמת Test, Staging ו-Prod. האפליקציות עבור

Observability I-Monitoring) ניטור

Kube-לא מספיק שהאפליקציה פועלת; צריך לדעת מה קורה בה בזמן אמת. בפרויקט הותקן הסטאק המלא של **Prometheus-Stack:**

- **Prometheus:** אוטומטית מטריקות מהקלאסטר, מהשרתים ומהפודים, ושומר אותם במסד (Scrape) סורק **Prometheus:** (Time-series) נתונים של סדרות-זמן

- ## בעיות צפויות ופתרונות

1. **Secrets Management (בעיה - ניהול סודות מאובטח):**

- ## 2. בעיה - התאמת כמות השרתים לעומס בצורה יעילה

- ### 3. Least Privilege - הרשאות ענן מאובטחות:

- ## תמונות קוד וצילומי מסך

1. GitHub Actions (CI)

2. ArgoCD ממשיך

3. Grafana דאשבורד

(של הקלאסטר CPU-הכנס צילום מסך של גרפנה מציג צריכת זיכרון ו)

4. דוגמה מקוד ה-GitHub Actions:

```
name: App CI

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Login to Amazon ECR
        uses: aws-actions/amazon-ecr-login@v1

      - name: Build, tag, and push image
        run: |
          docker build -t $ECR_REGISTRY/dogops-backend:${{ github.sha }}
            ./backend
          docker push $ECR_REGISTRY/dogops-backend:${{ github.sha }}
```

סיכום אישי

[הערה: פרק זה נועד לכתיבה אישית שלך. להלן טיוטה להשראה שתוכל לערוך ולהתאים לעצמך]

מודרניים. המעבר מלמידה DevOps פרויקט זה היווה עבורי קפיצת מדרגה משמעותית בהבנה ויישום של תהליכי תיאורטית של כלים נפרדים, לאינטגרציה מלאה של כולם יחד לכדי מערכת חיה, פועלת ונושמת בענן, היה מאתגר ומרתק כאחד.

ואת, (Terraform) למדתי כיצד מתמודדים עם תקלות בזמן אמת, כיצד מגדירים תשתיות בצורה מדויקת באמצעות קוד שהופכת את מאגר הגיט למקור האמת היחיד של המערכת ArgoCD באמצעות GitOps-החשיבות העצומה של גישת ה-External Secrets-לניהול שרתים ו Karpenter הכלים המתקדמים ששילבתי בפרויקט, כגון. (Single Source of Truth) הוא עוצמתי ומתרחב Kubernetes לאבטחת מידע, הוכיחו לי עד כמה האקו-סיסטם של

CI/CD-אני מסיים את הפרויקט בתחושת סיפוק רבה, עם ביטחון ביכולתי לתכנן, להקים ולנהל ארכיטקטורות ענן ו בסביבות עבודה מודרניות בשוק ההייטק.

סוף הספר.